

FLP 不可能性结果

任益驰 221900329

2026 年 6 月 27 日

目录

1	引言	2
1.1	主要问题	2
1.2	相关研究	2
1.3	核心结论	2
2	定义	3
2.1	术语	3
2.2	模型	4
3	证明	5
3.1	LEMMA 1	5
3.2	THEOREM 1	5
3.3	LEMMA 2	5
3.4	LEMMA 3	5
3.5	THEOREM 2	5
4	结论	5

1 引言

1.1 主要问题

分布式共识是分布式计算理论中最核心的问题之一。基本形式为：一组进程，每个进程持有一个输入值，它们需要通过彼此通信，最终全体一致地对一个输出值达成协议，且该输出值必须是某个进程的输入值。

形式化来说，一个共识协议需满足以下三条性质：

1. **一致性 (Agreement)**：所有节点达成一致的决议。
2. **有效性 (Validity)**：输出值必须是某个进程的输入值。也就是如果所有节点中的数据只有 0 和 1 两种，那么最后达成一致的决议一定是 0 和 1 中的一种，不会出现其他值。
3. **终止性 (Termination)**：所有正常运行的节点最终都能够做出决议。

共识问题是许多分布式系统的基础构件，一种广为人知的形式是事务提交问题，常见于分布式数据库系统。在该场景中，多个数据管理器进程需协同决定是否将某事务的结果永久写入数据库或全部撤销，共识问题是保证数据一致性的核心机制。因此，共识的可解性是一个既有理论深度又有工程意义的重要问题。

1.2 相关研究

在本文的工作之前，分布式系统领域已经对共识问题有了一定的认识。研究者们已经证明：在同步系统中，即使存在进程崩溃故障，共识问题也是可解的（如 Pease-Shostak-Lamport 的拜占庭将军问题）。然而，同步假设——即消息传递有已知的上界、进程步调以轮次同步推进——在现实系统中往往无法严格保证。

一个自然的问题是：如果我们放松同步假设，考虑异步系统（其中消息延迟无上界、进程之间没有公共时钟），共识问题是否仍然可解？

1.3 核心结论

Fischer、Lynch 和 Paterson 在论文《Impossibility of Distributed Consensus with One Faulty Process》中给出了一个令人震惊的结论：在消息系统可靠（没有丢包，消息不会被重复传递）的异步系统中，即使只允许一个进程崩溃故障，也无法保证共识问题的可解性。换句话说，在异步系统中，任何共识协议都无法同时满足一致性、有效性和终止性三条性质。

该结果被称为 FLP 不可能性结果，它揭示了分布式计算理论中的一个基本限制，并对后续的研究产生了深远影响。

2 定义

2.1 术语

Process: 在该论文中可以把它理解为分布式系统中的一个节点。每一个 process p 有一个 input register x_p , 可以理解为输入值, 只有 0 和 1 两种。此外 process p 还有一个 output register y_p , 可以理解为输出值, 有 b , 0 和 1 三种。即 $x_p \in \{0, 1\}, y_p \in \{b, 0, 1\}$ 。

Message system: 在该论文中可以把它理解为分布式系统中的一个通信网络, 连接了整个集群中的所有 process。整个系统中所有的 process 均共用这一个 message system, process 通过这个进行相互之间的通讯。

Asynchronous system: 异步系统。在该论文中对异步系统的核心假设是: 没有任何关于时间的信息可用。具体来说: 1) 没有任何关于消息传递延迟的假设。消息可以在任意时间被传递, 甚至可能永远不会被传递。2) 没有任何关于进程执行速度的假设。进程可以以任意速度执行, 甚至可能永远不会执行。3) 没有同步时钟。进程之间没有任何同步时钟, 无法通过时间来协调彼此的行为, 无法使用超时机制。4) 没有任何关于进程崩溃的假设。进程可能会崩溃, 但无法通过时间来判断一个进程是否已经崩溃, 无法区分“进程已崩溃”和“进程只是跑得非常慢”。

Internal state: 内部状态。文中将 Internal state 定义为一个元组, 即 $\text{internal state} = (\text{input register}, \text{output register}, \text{program counter}, \text{internal storage})$ 。

Configuration: 配置。一个 Configuration 包含了整个集群中所有 process 的 internal state 外加 message system。

Partially correct: 部分正确。一个共识协议是部分正确的, 需要满足两个条件: (1) 不存在任何一个可达到的 configuration 同时包含两种不同的决议值 (即不会出现矛盾)。(2) 对于每种可能的决议值 (0 和 1), 都存在某个可达到的 configuration 能够做出该决议 (即两种值都有可能被选中, 排除掉那种永远只输出 0 的平凡解)。简单来说, 部分正确只保证“不会错”, 但不保证“一定能出结果”。

Totally correct in spite of one fault: 单故障下完全正确。一个共识协议如果在部分正确的基础上, 还满足“每一个 admissible run 都是 deciding run”, 则称该协议是单故障下完全正确的。其中 admissible run 是指“至多一个进程故障, 且所有发往非故障进程的消息最终都能被收到”的运行, deciding run 是指“某个进程在该运行中做出了决策”的运行。简单来说, 完全正确比部分正确多了一个“活性”要求——只要故障不超过一个、消息最终能送达, 协议就必须在有限时间内给出结果。

Bivalent: 二值 (或“未锁定”)。对于一个 configuration C , 如果从 C 出发可以到达的 configuration 中, 既存在决议值为 0 的 configuration, 也存在决议值为 1 的 configuration, 则称 C 是 bivalent 的。也就是说, 系统从 C 出发, 既有可能最终决定为 0, 也有可能最终决定为 1, 尚未锁定方向。

Univalent: 单值（或“已锁定”）。与 bivalent 相对，如果从 C 出发可以到达的 configuration 中，所有可能路径都通向同一种决议值，则称 C 是 univalent 的。具体来说，若只通向 0 则称 0-valent，只通向 1 则称 1-valent。系统一旦进入 univalent 状态，命运的走向就已经确定了，尽管可能还没有进程实际做出决策。

2.2 模型

在这一部分，我来简单介绍论文建立的模型，其中包括：

1. 宽松的分布式环境

- (a) 不考虑 Byzantine 故障，即数据在传输过程不会被篡改。
- (b) 假设消息系统可靠，不会丢失数据包，也不会重复传输数据包。但是不保证数据传输的先后顺序，而且有时候会返回空 ($\emptyset$)。
- (c) 存在至多一个 process 会在某一个不确定的时间 dead。
- (d) 为了简化，论文假设所有 process 的 input register $x_p \in \{0, 1\}$ ，所有未崩溃的 process 的 output register $y_p \in \{0, 1\}$ 。而且对于正常的共识算法，肯定是要保证所有的非故障 process 必须选择相同值，但是论文在这里放宽了条件，只要求至少存在某个进程最终做出决策。
- (e) 前面提到 message system 可能返回空 ($\emptyset$)，但是论文假设了如果一个 process 不停从 message system 中接受 message，那它终究能够接收到本该发给它的 message。这样就模拟了网络的延迟，但是消息一定会送达。

2. 消息系统

Process 之间的通讯是通过消息系统实现的。message 的格式为 (p, m) , p 是 process, m 是 message 的内容。Message system 支持两种操作：

- (a) $\text{send}(p, m)$: 将消息 (p, m) 放入 message system 中。
- (b) $\text{receive}(p)$: 从 message system 中取出一个发往 p 的消息 (p, m) ，如果没有则返回空 ($\emptyset$)。

模型的核心特征：整个模型可以概括为一组确定型自动机 + 一个异步消息系统。其中：(1) 每个 process 是确定型的——给定相同的内部状态和输入消息，必然产生相同的后续行为。系统的所有非确定性来源于消息系统的调度，而非进程本身的行为。(2) 消息系统是唯一的非确定性来源——它可以选择何时投递哪条消息、何时返回空 ($\emptyset$)。(3) 两个层面通过“事件”交互：事件 $e = (p, m)$ 表示消息 m 被投递给进程 p ，触发 p 执行一步计算。论文中所有系统状态的演化都通过事件序列（即调度，schedule）来描述，而非通过时间轴。

3 证明

3.1 LEMMA 1

【待补充】

3.2 THEOREM 1

【待补充】

3.3 LEMMA 2

【待补充】

3.4 LEMMA 3

【待补充】

3.5 THEOREM 2

【待补充】

4 结论

【待补充】